ENUNCIADO TAREA 2
ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Fuerza bruta, programación dinámica y algoritmos greedy en grid pathfinding con murallas y minas.»

Fecha límite de entrega: 23 de noviembre de 2026

Equipo ALGORITMOS Y COMPLEJIDAD 2026-2

18 de octubre de 2026
20:23

Versión 0.0

Índice

1. Objetivos **3**

2. Definición del problema **3**

2.1. Regla greedy 4

3. Especificaciones **5**

3.1. Implementaciones 5

3.2. Mini-informe 8

4. Condiciones de entrega **8**

A. Casos de prueba **9**

A.1. Camino de máximo puntaje en una grilla con murallas y minas 9

1. Objetivos

Año 2026. En el marco de una ambiciosa misión de exploración planetaria desarrollada en la Universidad Técnica Federico Santa María, el vehículo explorador autónomo *Sansano-1* ha sido desplegado sobre una región hostil e inexplorada. El objetivo del rover es atravesar un sector cuadriculado desde su punto de inserción hasta la base de extracción, recolectando la mayor cantidad posible de información científica y recursos valiosos, mientras sortea formaciones rocosas impenetrables y antiguos campos minados que amenazan la integridad de su blindaje.

Desde el punto de vista computacional, el desafío consiste en resolver una variante enriquecida del problema de *grid pathfinding*: encontrar un camino en una grilla de $N \times M$ con murallas y minas que **maximice el puntaje** recolectado sin agotar las vidas disponibles. Para abordar este problema, se propone realizar un estudio experimental comparativo entre tres **paradigmas fundamentales de diseño de algoritmos**:

- FUERZA BRUTA: búsqueda exhaustiva de todas las trayectorias posibles.
- PROGRAMACIÓN DINÁMICA: optimización exacta explotando la subestructura óptima y superposición de subproblemas.
- GREEDY (Voraz): heurística de decisión local inmediata basada en la mejor opción visible en cada paso.

El desafío de esta tarea no radica en la dificultad teórica de diseñar los algoritmos, sino en **estudiar rigurosamente su comportamiento en la práctica**: analizar las curvas de tiempo de ejecución y consumo de memoria empíricos frente a las cotas asintóticas teóricas, y **cuantificar el costo de la miopía del algoritmo greedy** evaluando su brecha respecto a la solución óptima y su tasa de falla ante terrenos adversos.

2. Definición del problema

El terreno de exploración se modela como una cuadrícula G de N filas y M columnas. Cada celda (i, j), con $1 \le i \le N$ y $1 \le j \le M$, posee un **identificador de tipo** $t_{i,j} \in \{1, 0, -1\}$ y un **valor científico** entero $g_{i,j}$ (el cual puede ser positivo, cero o negativo):

- **Celda transitable** ($t_{i,j} = 1$): al transitar por ella, el rover suma $g_{i,j}$ al puntaje acumulado (valores positivos representan muestras geológicas valiosas; valores negativos corresponden a zonas de terreno inestable o radiación).
- **Muralla** ($t_{i,j} = 0$): formaciones rocosas o abismos infranqueables. El rover no puede ingresar a estas celdas bajo ninguna circunstancia.
- **Mina** ($t_{i,j} = -1$): artefacto explosivo enterrado. El rover puede ingresar a la celda, pero al hacerlo detona la mina: el blindaje absorbe el impacto consumiendo **una vida** y se restan P puntos al puntaje total por concepto de daños en el sistema. Su valor científico $g_{i,j}$ se ignora.

El rover *Sansano-1* inicia su trayecto en la celda de aterrizaje $(1,1)$ equipado con V unidades de blindaje (vidas) y tiene como destino la estación de extracción situada en la celda (N,M). Debido a las limitaciones mecánicas de su sistema de tracción y a la pendiente del terreno, desde una celda (i,j) solo se permiten dos maniobras de navegación:

- **Derecha / Este** (`R`): avanzar a la celda $(i, j+1)$, si $j < M$.
- **Abajo / Sur** (`D`): avanzar a la celda $(i+1, j)$, si $i < N$.

Si en algún momento las vidas del rover llegan a 0, este queda inoperativo y el camino se considera **inválido**. Por tanto, una trayectoria es válida si y solo si no atraviesa ninguna muralla y detona a lo más $V-1$ minas a lo largo de su recorrido. Se garantiza que la celda de partida $(1,1)$ y la de llegada (N,M) son siempre celdas normales seguras.

Formalmente, un camino es una secuencia de celdas $C = (c_1, c_2, \ldots, c_{N+M-1})$ con $c_1 = (1,1)$, $c_{N+M-1} = (N,M)$ y donde cada celda sucesiva c_{k+1} proviene de c_k mediante un movimiento válido. Denotando por minas(C) la cantidad de minas detonadas por el camino C, el puntaje total obtenido es:

$$s(C) = \left(\sum_{k=1}^{N+M-1} g_{c_k} \right) - P \cdot \text{minas}(C),$$

y el objetivo es encontrar un camino válido C^* que maximice dicho puntaje:

$$s(C^*) = \max_{C \text{ válido}} s(C).$$

Todo trayecto válido está conformado por exactamente $N-1$ movimientos hacia el sur (`D`) y $M-1$ movimientos hacia el este (`R`), totalizando $\binom{N+M-2}{N-1}$ caminos posibles en la cuadrícula.

Nota técnica: Debido a que el rover solo se desplaza hacia el este y hacia el sur, ningún camino puede visitar la misma celda en más de una ocasión; en consecuencia, los valores negativos no inducen ciclos infinitos. Además, sacrificar deliberadamente una unidad de blindaje al detonar una mina puede resultar una estrategia rentable si permite habilitar el paso hacia sectores de alto valor científico.

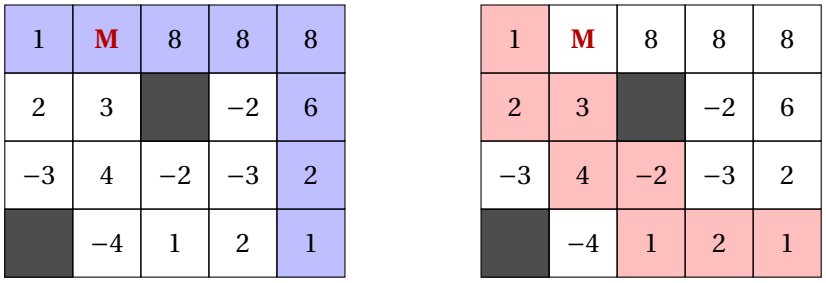
2.1. Regla greedy

Para que las soluciones heurísticas sean directamente comparables entre todos los estudiantes, el algoritmo GREEDY debe implementar con precisión el protocolo de navegación local establecido a continuación:

Regla greedy: Estando en la celda (i,j) con v vidas restantes, los sensores del rover evalúan como *candidatas* las celdas contiguas accesibles (derecha y abajo) que no sean murallas y que no sean minas si $v = 1$ (para evitar la destrucción inminente). La *ganancia neta* de una celda candidata es su valor g si es una celda normal, y $-P$ si es una mina. El rover avanza hacia la candidata de **mayor**

ganancia neta; en caso de empate, prioriza el movimiento hacia la **derecha**. Si ninguna de las dos celdas vecinas es transitable de forma segura, el algoritmo **falla** y se declara `SIN SOLUCION`.

Esta regla no garantiza alcanzar el óptimo global, e incluso puede conducir al rover a callejones sin salida. La Fig. 1 ilustra una situación donde la decisión voraz produce una solución subóptima frente a la estrategia global óptima.



Óptimo: puntaje 29 Greedy: puntaje 12

Figura 1: Grilla de ejemplo con $N = 4$, $M = 5$, $V = 2$ y $P = 5$. Las murallas se muestran en negro y las minas con la letra M. A la izquierda, el camino óptimo `RRRRDDD` (puntaje 29), que asume detonar una mina para acceder al corredor de alto valor superior y concluye con 1 vida. A la derecha, el camino obtenido por la regla greedy `DRDRDRR` (puntaje 12), que evade la mina de forma conservadora. Note que para $V = 1$, la solución óptima habría coincidido con la solución encontrada por la estrategia greedy.

3. Especificaciones

En esta sección se describen los pasos a seguir para completar la tarea, la cual consiste en desarrollar código en C++ «Implementaciones» y en realizar un mini-informe «Mini-informe». Se espera que cada uno de los pasos se realice de manera ordenada y siguiendo las instrucciones dadas.

3.1. Implementaciones

Nota: Abra este documento en algún lector de pdf que permita hipervínculos, ya que en este documento el texto en color azul suele indicar un hipervínculo.

(1) En caso de dudas, enviar preguntas **al foro de la Tarea 2**. En caso de cualquier modificación, se informará oportunamente por aula.

(2) Todo lo necesario para realizar la tarea se encuentra en el archivo `.zip` proporcionado a través de aula.usm.cl.

Nota: La estructura de archivos y el template del proyecto se encuentran disponibles en el material entregado.

(1) **Algoritmos.** Implementar en C++ **exactamente una solución por paradigma** para el problema definido en la «Definición del problema». Cada implementación debe retornar el **puntaje** del camino encontrado, el **camino** en sí (secuencia de movimientos `R` y `D`) y las **vidas restantes** al llegar a (N, M), o bien indicar que no encontró un camino válido (`SIN SOLUCION`).

Fuerza bruta. Enumerar exhaustivamente todos los caminos desde $(1, 1)$ hasta (N, M), calcular el puntaje de cada camino válido y retornar el de mayor puntaje. Se permite descartar un camino en cuanto se vuelva inválido (pisa una muralla o se queda sin vidas), pero no se permiten otras podas (por ejemplo, cotas sobre el puntaje acumulado) ni memorización: el objetivo es medir el costo computacional de la búsqueda exhaustiva pura.

Programación dinámica. Calcular el camino válido de mayor puntaje aprovechando la subestructura óptima del problema, considerando las vidas restantes como parte fundamental del estado. Puede implementarse de forma *top-down* (memorización) o *bottom-up* (tabulación), y se debe reconstruir la trayectoria óptima a partir de la información almacenada.

Greedy. Construir un camino aplicando en cada paso el protocolo local establecido en la «Regla greedy» de este enunciado.

Estructura de archivos requerida:

- `code/grid_pathfinding/algorithms/brute_force.cpp`
- `code/grid_pathfinding/algorithms/dynamic_programming.cpp`
- `code/grid_pathfinding/algorithms/greedy.cpp`

Las soluciones de FUERZA BRUTA y PROGRAMACIÓN DINÁMICA deben coincidir en el puntaje para todas las instancias en que ambas sean ejecutadas; verifique esto rigurosamente en su programa principal.

(2) **Mediciones de tiempo, memoria y calidad.** Implementar el programa principal en C++ y su respectivo `makefile` para ejecutar sistemáticamente los tres algoritmos sobre el conjunto de casos de prueba (ver «Casos de prueba»).

Los resultados de las mediciones deben almacenarse en archivos de texto dentro del directorio:

- `code/grid_pathfinding/data/measurements/`

Además del tiempo de ejecución y el consumo de memoria, para cada instancia se debe registrar el **puntaje** obtenido por cada algoritmo (o el indicador de falla), con el objetivo de evaluar la calidad de la solución greedy respecto al óptimo mediante:

- la **brecha** $\Delta = s(C^*) - s(C_{\text{greedy}}) \geq 0$, en los casos en que greedy encuentra un camino válido, y
- la **tasa de falla**, correspondiente a la proporción de casos en que greedy no logra alcanzar la meta.

Como los valores del terreno pueden ser negativos o cero, no se utiliza una razón porcentual como $s(C_{\text{greedy}})/s(C^*)$.

Archivos a entregar:

- `code/grid_pathfinding/grid_pathfinding.cpp`
- `code/grid_pathfinding/makefile`

Dado su costo factorial/exponencial, el algoritmo de FUERZA BRUTA solo debe ejecutarse en instancias con $N + M \leq 32$.

(3) **Generación de gráficos.** Implementar el programa en PYTHON encargado de leer las mediciones guardadas en el directorio de resultados, procesar los datos y generar los gráficos comparativos correspondientes (tiempo de ejecución vs tamaño, uso de memoria, brecha Δ y tasa de falla de greedy).

Las figuras generadas deben guardarse en formato PNG dentro del directorio:

- `code/grid_pathfinding/data/plots/`

Ubicación del script:

- `code/grid_pathfinding/scripts/plot_generator.py`

(4) **Documentación.** Documentar cada uno de los pasos anteriores:

- Completar el archivo `README.md` del directorio `code/`.
- Documentar al inicio de cada archivo fuente las referencias bibliográficas, fuentes de consulta y decisiones de diseño adoptadas en la implementación de cada algoritmo.

3.2. Mini-informe

Generar un mini-informe en LaTeX que contenga una introducción breve, los resultados obtenidos (tablas y gráficos) con una discusión sobre ellos, y una conclusión breve. La discusión debe abordar, al menos, la relación entre el tiempo y la memoria medidos y la complejidad teórica de cada paradigma, y la calidad de las soluciones del algoritmo greedy (brecha Δ y tasa de falla) según el tamaño, tipo y dominio de la grilla. En el material entregado podrá encontrar el Template en el directorio report que deben utilizar:

```
Template disponible en: report/ dentro del material entregado
```

Las indicaciones se encuentran en el archivo README.md y en la plantilla de LaTeX.

4. Condiciones de entrega

(1) La tarea es individual, sin excepciones.

(2) La entrega debe realizarse vía aula.usm.cl, subiendo un archivo `.zip` que contenga todo el contenido de su proyecto (código, informe, datos, gráficos, etc.).

(3) Si se utiliza algún código, idea, o contenido extraído de otra fuente, este debe ser citado en el lugar exacto donde se utilice.

(4) Asegúrese que todas sus entregas tengan sus datos completos: número de la tarea, ramo, semestre, nombre y rol. Puede incluirlas como comentarios en sus fuentes LaTeX (en TeX comentarios son desde % hasta el final de la línea) o en posibles programas. Anótese como autor de los textos.

(5) Si usa material adicional al discutido en clases, detállelo. Agregue información suficiente para ubicar ese material (en caso de no tratarse de discusiones con compañeros de curso u otras personas).

(6) La fecha límite de entrega es el día 23 de noviembre de 2026.
NO SE ACEPTARÁN TAREAS FUERA DE PLAZO.

(7) Nos reservamos el derecho de llamar a interrogación sobre algunas de las tareas entregadas. En tal caso, la nota de la tarea será la obtenida en la interrogación.
NO PRESENTARSE A UN LLAMADO A INTERROGACIÓN SIN JUSTIFICACIÓN PREVIA SIGNIFICA AUTOMÁTICAMENTE NOTA 0.

A. Casos de prueba

A.1. Camino de máximo puntaje en una grilla con murallas y minas

Entrada: Leer una grilla G desde el archivo `{n}_{m}_{t}_{d}_{s}.txt`, con el siguiente formato:

- La primera línea contiene cuatro enteros `N M V P`: número de filas, número de columnas, vidas iniciales y puntaje que se pierde al pisar una mina, respectivamente.
- Las siguientes N líneas contienen M enteros separados por espacio, correspondientes a los **identificadores de tipo** $t_{i,j}$ de cada celda: `1` celda normal, `0` muralla y `-1` mina.
- Las siguientes N líneas contienen M enteros separados por espacio, correspondientes a los **valores** $g_{i,j}$ de cada celda. En las murallas y minas el valor se escribe como `0` y se ignora.

Los parámetros de los casos de prueba son:

- n y m hacen referencia al número de filas N y de columnas M de la grilla, con $N \in \{2^2, 2^3, 2^4, 2^6, 2^8, 2^{10}\}$ y $M \in \{N, 2N\}$. El algoritmo de FUERZA BRUTA solo se ejecuta cuando $N + M \leq 32$.
- t hace referencia al tipo de grilla, y pertenece al conjunto $T = \{\text{libre}, \text{murallas}, \text{minas}, \text{mixta}\}$. Sea $K = \lfloor \sqrt{N \cdot M} \rfloor$:
  - *libre*: todas las celdas son normales.
  - *murallas*: se ubican K murallas en posiciones aleatorias.
  - *minas*: se ubican K minas en posiciones aleatorias.
  - *mixta*: se ubican K murallas y K minas en posiciones aleatorias.
- d hace referencia al dominio de los valores de las celdas normales, $d = \{D10, D1000\}$, donde $D10$ implica que el dominio es $\{-10, -9, \ldots, 9, 10\}$ y $D1000$ que el dominio es $\{-1000, -999, \ldots, 999, 1000\}$. La penalización por mina es $P = 10$ y $P = 1000$, respectivamente.
- s hace referencia a la muestra aleatoria (o caso de prueba) y pertenece al conjunto $S = \{a, b, c\}$.

En todos los casos de prueba se usa $V = 3$. El generador garantiza que las celdas $(1, 1)$ y (N, M) son normales y que existe al menos un camino válido; aun así, el algoritmo greedy puede fallar.

Salida: Para cada algoritmo $a \in \{\text{bruteforce}, \text{dp}, \text{greedy}\}$, escribir en el archivo:
`{n}_{m}_{t}_{d}_{s}_{a}_out.txt`

- en la primera línea, el puntaje del camino encontrado;
- en la segunda línea, la secuencia de $N + M - 2$ movimientos (`R` o `D`) que lo describe;
- en la tercera línea, las vidas restantes al llegar a (N, M).

Si el algoritmo no encuentra un camino válido, el archivo debe contener únicamente la línea `SIN SOLUCION`. Por ejemplo, para la grilla de la Fig. 1, la salida del algoritmo de programación dinámica sería:

```
29
RRRRDDD
1
```

Directorios de datos y scripts asociados:

- Instancias de entrada: `code/grid_pathfinding/data/grid_input/`
- Salidas generadas: `code/grid_pathfinding/data/grid_output/`
- Script generador de grillas: `code/grid_pathfinding/scripts/grid_generator.py`